https: / / github.com / fhlsteven / CLR_via_CSharp / blob / main / chapters / ch23_AssemblyLoaingReflection.md 我现在在学习《CLR via C#》这本书的第23章，内容如链接

用户:

https://github.com/fhlsteven/CLR_via_CSharp/blob/main/chapters/ch23_AssemblyLoaingReflection.md

我现在在学习《CLR via C#》这本书的第23章，内容如链接所示，你总结一下当前章的各节内容，我看你的总结学习就行了，不看文章了。

元宝:

根据你给的 GitHub 链接，这是《CLR Via C#》第四版的第23章「程序集加载和反射」，对应第3版的小节结构如下 ：

- 23.1 程序集加载
- 23.2 使用反射构建动态可扩展应用程序
- 23.3 反射的性能（含 23.3.1 ~ 23.3.4 ）
- 23.4 设计支持加载项的应用程序
- 23.5 使用反射发现类型的成员（含 23.5.1 ~ 23.5.6 ）

下面按节给你提炼核心内容，你可以直接照这个学。

23.1 程序集加载

JIT 编译器把 IL 编译成本地代码时，会查看 IL 引用的类型，然后借助程序集的 `TypeRef` 和 `AssemblyRef` 元数据表来判断"哪个程序集定义了这个类型"。CLR 内部通过 `System.Reflection.Assembly` 的静态方法 **`Load`** 来完成加载——它是 CLR 版的 `LoadLibrary` 。

常用的加载方法：

- **`Assembly.Load(assemblyRef)`**：传入 `AssemblyName` 或程序集字符串。CLR 会应用版本绑定重定向策略，先查 GAC，再到应用基目录、私有路径子目录、 `codebase` 位置查找。**这是动态可扩展应用加载程序集的首选方式** 。
- **`Assembly.LoadFrom(path)`**：加载指定路径（甚至可以是 URL ）的程序集。内部流程是先调用 `AssemblyName.GetAssemblyName` 拿到程序集标识，再调用 `Assembly.Load` ；若按标识找不到，则按实参路径加载 。

- `Assembly.LoadFile`：可从任意路径加载，且允许把"相同标识"的程序集多次加载到同一个 AppDomain 。
- `Assembly.ReflectionOnlyLoad / ReflectionOnlyLoadFrom`：只加载程序集用于分析元数据，**禁止其中任何代码执行** 。
- **弱命名程序集**调用 `Load` 时，不会走版本绑定重定向，也不会查 GAC 。

> 💡 关于依赖 DLL：可以把依赖的 DLL 作为"嵌入资源"打进 EXE，部署时只需分发一个文件；运行时在 `AppDomain.CurrentDomain.AssemblyResolve` 事件中登记回调，从资源流中读出字节数组再调用 `Assembly.Load(byte[])` 加载 。

23.2 使用反射构建动态可扩展应用程序

元数据本质上是一系列表（类型定义表、字段定义表、方法定义表等）。利用 `System.Reflection` 命名空间下的类型，可以在运行时"解析"这些元数据表，从而发现并使用编译时完全不了解的类型 。

这种机制的经典场景是**宿主应用程序 + 加载项（add-in）**：宿主方在编译时并不知道插件的具体类型，需要在运行时动态发现并调用。

> ⚠️ 注意：要求反射查找名为 `int` 的类型会通过编译，但运行时返回 `null`，因为 CLR 只认 `System.Int32`，不认 `int` 这个 C# 关键字别名 。

23.3 反射的性能

反射有两个核心缺点：**编译时无法保证类型安全** + **速度慢** 。

慢在哪里：

1. 类型、成员都用字符串标识，运行时要在元数据表里做大量字符串搜索（通常还不区分大小写）
2. 用反射调用方法时，实参要先打包成数组 → 内部解包到线程栈 → CLR 检查实参类型 → 检查调用者安全权限

书中给出**绕开反射调用**的两条推荐路径 ：

- **让类型从编译时已知的基类型派生** → 运行时构造实例，引用放到基类变量中，调用基类虚方法
- **让类型实现编译时已知的接口** → 运行时构造实例，引用放到接口变量中，调用接口方法

> 📌 作者个人更偏好**接口方案**，因为基类方案会限制开发人员选择最合适的基类；但涉及版本控制时基类方案更好——往基类加成员，派生类自动继承，而接口加成员则要求所有实现者改代码重编译。**强烈建议把接口/基类放到独立的程序集中**，以缓解版本控制问题（详见 23.4）。

23.3.1 发现程序集中定义的类型

最常用 API 是 `Assembly.ExportedTypes` 属性，返回程序集中所有 `public` 且对外可见的类型 。

23.3.2 类型对象的准确含义

程序集中每个类型只有一个 `Type` 对象，所以可以用 `==` 判断两个 `Type` 引用是否指向同一类型 。

23.3.3 构建 Exception 派生类型的一个层次结构

通过一个具体例子演示如何用反射发现类型并构建异常类型的层次结构（用于动态展示/处理）。

23.3.4 构造类型的实例

FCL 提供多种机制 ：

- **`System.Activator.CreateInstance`**：最简单，可传 `Type` 对象或类型字符串；传字符串的版本返回 `ObjectHandle`（需调用 `Unwrap()` 具体化）
- **`System.AppDomain` 实例方法 `CreateInstanceFrom`**：指定在哪个 AppDomain 中实例化
- **`System.Reflection.ConstructorInfo.Invoke`**：拿到构造器信息后调用
- **`Array.CreateInstance`**：创建数组
- **`MethodInfo.CreateDelegate`**：创建委托

23.4 设计支持加载项的应用程序

这一节讲如何把"宿主 + 插件"的架构落地。**关键思想**：把公共接口（或基类）放到一个**独立的、版本稳定的程序集**中，宿主和加载项都引用它。这样加载项可以独立于宿主发布，宿主在运行时通过 `Assembly.LoadFrom` 加载插件程序集、通过接口/基类多态调用。

23.5 使用反射发现类型的成员

这里是反射操作最细粒度的部分。

23.5.1 发现类型成员

通过 `Type.GetMembers()` 等方法获取 `MemberInfo` 派生对象（如 `FieldInfo`、`MethodInfo`、`PropertyInfo` 等）。

23.5.2 BindingFlags：筛选返回的成员种类

用 `BindingFlags` 枚举来控制要返回哪些成员（public/private、static/instance、declared only 等）。

23.5.3 发现类型的接口

通过 `Type.GetInterfaces()` 获取类型实现的接口列表。

23.5.4 调用类型的成员

调用成员的几种方式，性能从低到高 ：

1. **`Type.InvokeMember`** – 最慢，每次都要重新绑定
2. **`FieldInfo / MethodInfo / PropertyInfo` 等** – 一次绑定，多次调用，性能更好
3. **创建委托引用成员** – 比方式 2 更快
4. **C# 的 `dynamic` 基元类型** – 简化语法

23.5.5 一次绑定，多次调用

强调用 `MethodInfo` / `FieldInfo` 等做"一次绑定、多次调用"，避免每次调用都重复绑定开销。

23.5.6 使用绑定句柄来减少进程的内存耗用

使用 **运行时句柄** `RuntimeTypeHandle`、`RuntimeFieldHandle`、`RuntimeMethodHandle` 来减少进程的内存开销。当需要在内存中缓存大量类型/成员信息时，存句柄比存 `Type` / `MemberInfo` 对象更省内存。

一图掌握全章主线

> 程序集怎么加载进来（23.1）→ 用反射动态发现类型、构建可扩展应用（23.2）→ 但反射很慢，要学会用接口/基类规避直接反射调用（23.3）→ 落地到加载项架构设计（23.4）→ 细粒度地发现并调用类型成员，并用 BindingFlags、一次绑定、绑定句柄等手段优化（23.5）

学完这一章你应该掌握的硬技能：**`Assembly.Load/LoadFrom` 的区别与适用场景、用接口而非反射调用来提升性能、`Activator.CreateInstance` 的各种重载、`BindingFlags` 的用法、一次绑定多次调用与绑定句柄优化内存。**

如果你想深入某一节（比如 23.4 加载项设计的完整代码骨架，或 23.5.6 绑定句柄的具体用法），告诉我哪一节，我给你拆到可抄代码的程度。

▾ 引用 9 篇资料作为参考

1. CLR via C#(第3版)
2. 《CLR via C#》读书笔记 之 程序集加载和反射
3. CLR Via C# 读书笔记-第23章(程序集加载和反射)
4. 《CLR via C#》笔记:第4部分 核心机制(3)
5. CLR via C# (第3版) - Jeffrey Richter 著 - 读书笔记 - 爱码网
6. CLR Via C# 3rd 阅读摘要 -- Chapter 23 – Assembly Loading and Reflection
7. CLR via C#:程序集加载和反射_金朝的博客
8. 反射 (C/CLI)
9. Jeffrey Richter: Excerpt #2 from CLR via C#, Third Edition

用户:

你提到的“元数据”不太懂，其是什么？和程序集有什么关系？

元宝:

好问题，"元数据"是 CLR 里非常底层但贯穿全书的核心概念，搞懂它，23 章的很多内容就顺了。

元数据是什么

简单来说，**元数据就是描述数据的数据**——在这里，它是**描述程序集中"有哪些类型、每个类型有哪些成员、这些方法参数是什么、引用了哪些其他程序集"等信息的二进制数据**。

打个比方：

- 你写了一份 **Word 文档**（= 程序集）
- 文档里有正文内容（= IL 中间语言代码，真正干活的指令）

- 但 Word 文档还有一份**隐藏的属性信息**：作者是谁、创建时间、字数多少、用了哪些样式……（= **元数据**）

元数据就是程序集的"自描述说明书"，它告诉 CLR：

- 这个程序集里**定义了哪些类型**（类、接口、结构体、枚举、委托）
- 每个类型里**有哪些字段、方法、属性、事件**
- 这些成员的**访问修饰符、返回类型、参数类型**
- 这个程序集**依赖/引用了哪些其他程序集**

元数据的物理形态：表

在 PE 文件格式里，元数据不是杂乱存放的，而是组织成一系列**表（table）**：

表名	存的内容
ModuleDef	模块定义
TypeDef	类型定义（类、接口等）
FieldDef	字段定义
MethodDef	方法定义
MemberRef	成员引用
AssemblyRef	程序集引用
TypeRef	类型引用
AssemblyDef	当前程序集的定义
...	共几十张表

每张表由若干**记录（row）**组成，每条记录有固定的列。比如 `TypeDef` 表的每一行描述一个类型：它的名称、命名空间、基类、标志位等 。

元数据 vs. 程序集：包含关系

这是关键，记住这句话：

> **程序集（Assembly）= 清单（Manifest）+ 元数据（Metadata）+ IL 代码 + 资源（Resources）**

也就是说，**元数据是程序集的内部组成部分之一**，二者是"整体与局部"的关系 ：

```
程序集 (.dll / .exe)
├── 清单 (Manifest)          ← 程序集的"身份证"：名称、版本、文化、公钥、入口点
├── 元数据 (Metadata)        ← 类型的详细描述（一堆表）
├── IL 代码                  ← 方法体里的实际指令
└── 资源 (Resources)         ← 图片、字符串表等嵌入资源
```

- **清单**告诉你"我是谁、我依赖谁"
- **元数据**告诉你"我里面有什么类型、这些类型长什么样"

- **IL** 才是真正被 CPU 执行的代码（经 JIT 编译后）

元数据在 23 章里扮演的角色

回到你正在学的第 23 章，所有反射操作**本质上都是在"读元数据表"**：

1. 程序集加载时，CLR 读元数据

当你写 `Assembly.Load("SomeLib")` 时，CLR 找到 `SomeLib.dll`，**读取它的 AssemblyRef 元数据表**，看它依赖哪些程序集，然后递归加载那些依赖项 。这就是 23.1 节讲的程序集加载机制的数据基础。

2. 反射就是遍历元数据表

`Type.GetMembers()` 本质是扫描 `TypeDef` 表 + `MethodDef` / `FieldDef` 等表，把每行记录包装成 `MemberInfo` 对象返回给你。这就是为什么 23.3 节说"反射慢"——它要做大量的**字符串匹配**去查这些表。

3. JIT 编译时，CLR 读 TypeRef + AssemblyRef

上一轮我们提到：JIT 编译 IL 时遇到一个类型引用，会查 IL 的 `TypeRef` 表和 `AssemblyRef` 表，判断"这个类型定义在哪个程序集里"，然后触发那个程序集的加载 。这条链路全靠元数据驱动。

用一段 IL 直观感受

用 `ildasm` 或 `dotnet-ildasm` 打开任何一个 DLL，你会看到类似：

```
// 元数据里的 TypeDef 表（节选）
.class public auto ansi beforefieldinit MyApp.Person
    extends [mscorlib]System.Object
{
    // 字段（FieldDef 表的记录）
    .field private string '<Name>k__BackingField'

    // 方法（MethodDef 表的记录）
    .method public hidebysig specialname instance string
        get_Name() cil managed { ... }
}
```

上面这段**不是代码，而是元数据的人可读形式**——它描述了 `Person` 类的结构。而 `get_Name()` 方法 `{ ... }` 花括号里那堆才是 IL 代码。

为什么元数据这么重要

1. **语言互操作**：C# 写的类，VB.NET 能直接用，因为元数据是语言中立的——任何 .NET 语言编译后都生成统一的元数据格式
2. **无需头文件/IDL**：传统 C++ 要 `#include` 头文件才知道别人定义了什么；.NET 里直接读程序集的元数据就行，所以**不需要头文件**
3. **反射的基础**：运行时动态发现类型全靠它
4. **GC 的根基**：GC 要判断一个对象是否可回收，需要知道"哪些字段是引用类型"——这信息就在元数据里
5. **序列化、IDE 智能提示、调试器**……全都依赖元数据

一句话串起来：程序集是 .NET 的部署单元，元数据是程序集里的"类型说明书"（一系列二进制表），IL 是真正干活的代码，而反射就是运行时去读这份"说明书"的过程。23 章讲的"程序集加载"和"反射"，本质都是围绕元数据转——加载是为了让 CLR 读到元数据，反射是为了让你在代码里读到元数据。

用户:

https://github.com/fhlsteven/CLR_via_CSharp/blob/main/chapters/ch22_CLRHostingAndAppDomain.md

这个是22章的链接，把23章和22章串联起来。

元宝:

这两章其实是**"运行时底座"和"运行时操作"**的关系——第 22 章搭舞台，第 23 章在舞台上表演。串起来看，你会发现 23 章里几乎所有 API 的背后，都站着 22 章的 AppDomain。

两章的主线拼接

```
Windows 进程
   └─ CLR（通过寄宿加载，第22章 22.1）
         ├─ 默认 AppDomain（第22章 22.2）
         │      └─ 程序集 A ──┐
         ├─ 插件 AppDomain 1 ├← 程序集在这里加载（第23章 23.1）
         │      └─ 程序集 B ──┤   Assembly.Load / LoadFrom
         └─ 插件 AppDomain 2 ┘
                └─ 程序集 C ──┘
                      ↓
            用反射发现类型、构造实例（第23章 23.2~23.5）
                      ↓
            跨 AppDomain 通信（第22章 跨越边界）
                      ↓
            AppDomain.Unload 卸载（第22章 22.3）
```

第 22 章告诉你：CLR 是怎么被宿主进程加载进来的，以及加载进来后创建了 AppDomain 这个"逻辑容器"来隔离一组程序集 。
第 23 章告诉你：程序集具体怎么被加载进 AppDomain，以及加载后如何用反射去"解剖"它 。

所以两章是**"容器"与"内容物"**的关系：22 章讲容器，23 章讲往容器里装什么、怎么装、装完怎么用。

第 22 章核心回顾（3 个关键概念）

1. CLR 寄宿（CLR Hosting）

CLR 本身是一个 COM 服务器（DLL），任何 Windows 进程都可以把它加载进来当"运行时管家"。宿主通过 `ICLRRuntimeHost` 接口做五件事 ：

- 设置宿主管理器（内存、程序集加载、安全、线程同步）
- 初始化并启动 CLR

- 加载程序集并执行代码
- 管理运行时生命周期
- 停止 CLR

典型宿主：ASP.NET（每个虚拟目录一个 AppDomain）、SQL Server（执行托管存储过程）、以及你自己写的宿主。

2. AppDomain：程序集的逻辑容器

CLR 启动时自动创建一个**默认 AppDomain**，它随进程死亡而销毁。除此之外，宿主可以通过托管 API 或 COM 接口让 CLR 创建更多 AppDomain。

AppDomain 的四大特性 ：

特性	说明
隔离	一个 AppDomain 里的代码不能直接访问另一个 AppDomain 创建的对象
可卸载	可以整体卸载一个 AppDomain（但**不能单独卸载某个程序集**）
可单独保护	创建时可施加权限集，限制其中代码的权限
可单独配置	搜索路径、版本绑定重定向、卷影复制、加载器优化等都按 AppDomain 配置

> 💡 关键限制：**CLR 不支持从 AppDomain 中单独卸载一个程序集**。要回收程序集占用的内存，唯一的办法是卸载整个 AppDomain。这条限制直接决定了第 23 章插件架构的设计思路。

3. 跨越 AppDomain 边界

两个 AppDomain 中的对象不能直接互相引用，必须通过：

- **按引用封送（Marshal-by-Reference）**：类型继承 `MarshalByRefObject`，跨域拿到的是代理对象
- **按值封送（Marshal-by-Value）**：对象可序列化，复制一份到目标 AppDomain

第 23 章在第 22 章基础上的延展

现在我们把 23 章的每个小节，都挂到 22 章的 AppDomain 上下文中去看，你会发现一切都很顺：

23.1 程序集加载 → 加载进哪个 AppDomain？

`Assembly.Load` 的文档明确说：JIT 编译器根据 IL 的 `TypeRef` 和 `AssemblyRef` 元数据表确定类型所属的程序集，**然后把该程序集加载到当前 AppDomain 中**。

也就是说：

- `Assembly.Load("Plugin.dll")` 是把 Plugin.dll **加载到调用这段代码所在的 AppDomain**
- 如果你想加载到别的 AppDomain，要用 `AppDomain` 实例的 `Load` 方法——但 23 章明确指出这个方法设计上是给非托管宿主调用的，**托管代码开发人员应优先用 `Assembly.Load`**

> ⚠️ 23 章特别提醒：不要随便调 `AppDomain.Load`。因为它返回 `Assembly` 对象时，`Assembly` 类不继承 `MarshalByRefObject`，必须**按值封送**回调用方的 AppDomain，这会导致 CLR 用调用方 AppDomain 的策略去

> 重新定位程序集，很可能抛出 `FileNotFoundException` 。

所以正确的插件加载姿势是（结合两章）：

1. 用 22 章的方式创建新的 AppDomain：`AppDomain.CreateDomain("PluginDomain", null, setup)`
2. 在新 AppDomain 中创建宿主自己的 `MarshalByRefObject` 派生类型实例
3. 通过这个代理对象，在新 AppDomain **内部**调用 `Assembly.LoadFrom` 加载插件程序集
4. 插件类型构造实例后，转型为编译时已知的接口，通过代理暴露给默认 AppDomain

23.2 动态可扩展应用 → 为什么需要 AppDomain 隔离？

23 章构建插件系统时，如果只是用 `Assembly.Load` 把插件加载到默认 AppDomain，会遇到一个致命问题：**插件程序集永远无法从内存中卸载**（只有卸载 AppDomain 才能卸载程序集）。

这正是 22 章 AppDomain 的价值所在——为每个插件创建独立的 AppDomain，不用了就 `AppDomain.Unload` 整体卸掉，干净利落 。

23.3 反射的性能 → 接口/基类方案与跨域通信衔接

23.3 节强调：用反射直接调用成员很慢，应该**把对象转型为编译时已知的接口或基类**，通过多态调用 。

这个建议在跨 AppDomain 场景下**更重要**：

- 如果你的插件类型实现了宿主已知的接口，那么跨域通信时可以直接把接口引用返回
- 如果插件类型是 `MarshalByRefObject` 的派生类，那么跨域拿到的是透明代理，调用接口方法性能尚可
- 反之，如果只能用反射调用，每次跨域调用都要走封送 + 反射双重开销

23.4 设计支持加载项的应用程序 → 22+23 的完整拼图

这一节实际上是两章知识的综合应用。一个工业级插件系统的标准做法 ：

```
默认 AppDomain（宿主）
    │
    │  1. 创建独立 AppDomain
    ▼
插件 AppDomain
    │
    │  2. 创建宿主的 MarshalByRefObject 派生类型实例
    ▼
宿主代理对象（在插件 AppDomain 中）
    │
    │  3. Assembly.LoadFrom 加载插件 DLL
    │  4. Activator.CreateInstance 构造插件类型
    │  5. 转型为已知接口 IPlugin
    ▼
IPlugin 实例
    │
    │  6. 通过代理返回接口引用给默认 AppDomain
```

```
        ▼
宿主代码通过 IPlugin 接口调用插件
```

关键设计点：

- **公共接口程序集**：宿主和插件都引用同一个独立的接口程序集（23.4 节强调要独立出来做版本控制）
- **跨域通信**：宿主在插件 AppDomain 中创建自己的 `MarshalByRefObject` 派生类型，通过它与插件交互
- **卸载**：`AppDomain.Unload(pluginDomain)` 一次性回收插件占用的所有资源

23.5 反射发现成员 → 主要用在工具场景

这一节的细粒度反射（GetMembers、BindingFlags、MethodInfo.Invoke 等）主要面向**开发工具和分析程序**（ILDasm、PEVerify 等），而不是运行时插件调用。

在日常插件架构中，你很少需要走到 23.5 这一步——因为你在 23.4 已经通过接口转型拿到了编译时已知的类型引用，直接调用即可，不需要再用反射去"抠"成员。

一张表看清两章的对应关系

第 22 章概念	第 23 章对应物	串联关系
CLR 寄宿	JIT 编译时读 AssemblyRef 元数据表触发加载	寄宿启动 CLR → CLR 才有能力加载程序集
AppDomain 是程序集容器	`Assembly.Load` 把程序集加载进 AppDomain	容器决定了程序集的隔离边界
AppDomain 不能单独卸载程序集	插件 DLL 加载后常驻	所以要卸载插件 → 必须卸载整个 AppDomain
跨 AppDomain 通信：MarshalByRefObject	插件类型转型为已知接口	接口 + MarshalByRefObject = 跨域调用的高性能方案
AppDomain 可单独配置加载策略	`Assembly.LoadFrom` 按路径加载	每个 AppDomain 有自己的搜索路径和版本重定向策略
`AppDomain.Unload`	配合 `AssemblyResolve` 事件	卸载时触发清理，嵌入式资源程序集也随 AppDomain 消失

学习建议：把两章当一个大章节读

> 📌 **22 章是"地"，23 章是"图"**。只看 23 章，你会觉得程序集加载就是 `Assembly.Load` 一行事；串起 22 章后，你会明白这行代码背后有 CLR 寄宿、有 AppDomain 隔离边界、有跨域通信的封送开销、有"不能单独卸载程序集"的限制。

推荐的阅读顺序：

1. 先建立"进程 → CLR → AppDomain → 程序集"的四层模型（22 章）
2. 再看程序集在这个模型里怎么被加载进来（23.1）
3. 然后看加载进来后怎么用反射发现类型（23.2~23.3）
4. 接着看如何把"AppDomain 隔离"和"反射加载"组合起来做插件系统（23.4 + 22 章的跨域通信）
5. 最后看细粒度反射操作（23.5），知道它的主战场是工具开发而非运行时插件

如果你按这个顺序理解，23.4 节那段"宿主在新 AppDomain 中创建自己的 MarshalByRefObject 派生类型，让它去加载插件"的代码就不抽象了——它就是两章知识的接缝处。

要不要我把 23.4 节的插件系统代码骨架完整写出来？包含：宿主端、接口程序集、插件端、跨 AppDomain 通信、卸载回收的完整可运行示例。这样你能直观看到两章是怎么"焊接"在一起的。

▾ 引用 9 篇资料作为参考

1. CLR第二十二章—CLR寄宿与AppDomain(CLR Hosting and App Domains)
2. CLR via C# - Chapter 23 Assembly Loading and Reflection
3. 《CLR via C#》读书笔记 之 CLR寄宿和AppDomain
4. CLR via C# - Chapter 22 CLR Hosting and AppDomains
5. .NET:CLR via C#:CLR Hosting And AppDomains
6. Jeffrey Richter: Excerpt #2 from CLR via C#, Third Edition
7. CLR Hosted Environment
8. 第IV部分核心机制-第22章CLR寄宿和AppDomain-《CLRviaC#第4版》
9. Application Domains and Assemblies

用户:

你上面提到的"这种机制的经典场景是宿主应用程序 + 加载项（add-in）：宿主方在编译时并不知道插件的具体类型，需要在运行时动态发现并调用。"，不太懂

元宝:

没问题，这个"宿主 + 加载项"是 23 章的核心场景，我用大白话 + 具体例子给你讲透。

用 VS Code 来理解：你自己就在用"宿主 + 加载项"

你每天用的 VS Code 就是一个**宿主（Host）**，你安装的那些插件（C# 扩展、Python 扩展、GitLens 等）就是**加载项（Add-in）**。

微软开发 VS Code 的时候，**根本不知道你将来会装什么插件**——可能是有人写个"彩虹猫"插件，也可能是微软自己后来开发的 C# 扩展。所以：

- **编译时（开发 VS Code 时）**：微软的程序员写代码，他们**不可能引用**"彩虹猫插件"的类型，因为这个插件当时还不存在
- **运行时（你启动 VS Code 后）**：VS Code 扫描 `extensions` 文件夹，发现了你装的插件 DLL，**动态加载**进来，然后调用插件里实现好的功能

这就是"宿主方在编译时并不知道插件的具体类型，需要在运行时动态发现并调用"。

拆解"编译时不知道"是什么意思

假设没有插件机制，硬编码写死：

```
// ❌ 假设 VS Code 团队在代码里直接调用插件
var rainbowCat = new RainbowCatPlugin();  // 编译错误！RainbowCatPlugin 类根本不存在
rainbowCat.Activate();
```

这行代码**编译不过**，因为 `RainbowCatPlugin` 这个类在 VS Code 开发时还没被发明出来。

所以宿主必须换一种方式——**不依赖具体类型，只依赖一个"契约"（接口）**：

```
// ✅ 宿主定义一个接口（契约）
public interface IPlugin
{
    void Activate();   // 所有插件都必须实现这个方法
    string Name { get; }
}

// ✅ 宿主加载插件时，只认 IPlugin 接口，不认具体类型
public class PluginHost
{
    public void LoadAllPlugins(string pluginsFolder)
    {
        foreach (var dllPath in Directory.GetFiles(pluginsFolder, "*.dll"))
        {
            // 1. 动态加载程序集（23.1 节的内容）
            Assembly asm = Assembly.LoadFrom(dllPath);

            // 2. 在程序集中查找实现了 IPlugin 接口的类型（23.2 节的内容）
            foreach (var type in asm.ExportedTypes)
            {
                if (typeof(IPlugin).IsAssignableFrom(type))
                {
                    // 3. 动态创建实例（23.3.4 节的内容）
                    IPlugin plugin = (IPlugin)Activator.CreateInstance(type);

                    // 4. 通过接口调用，无需知道具体类型
                    Console.WriteLine($"加载插件: {plugin.Name}");
                    plugin.Activate();
                }
            }
        }
    }
}
```

注意第 4 步：宿主拿到的变量类型是 `IPlugin`——**这是一个编译时已知的接口**，宿主代码可以放心地调用 `.Activate()`、`.Name`。至于背后是哪个具体类（`RainbowCatPlugin`、`CSharpExtension`、`GitLens`），宿主**完全不关心**。

插件的写法（第三方开发者视角）

彩虹猫插件的开发者会这样写：

```
// 第三方开发者写的插件，引用宿主提供的 IPlugin 接口
public class RainbowCatPlugin : IPlugin
{
    public string Name => "彩虹猫";

    public void Activate()
    {
        Console.WriteLine("🌈🐱 喵呜~");
    }
}
```

编译成 `RainbowCatPlugin.dll`，丢进 VS Code 的 `extensions` 文件夹。宿主启动时扫到它，按上面的流程加载、实例化、调用。

关键点：宿主和插件**共享同一个 `IPlugin` 接口定义**，但彼此独立编译、独立部署。宿主编译时根本不知道 `RainbowCatPlugin` 的存在，但它依然能在运行时成功调用它。这就是"动态发现并调用"。

为什么必须这么做？三个核心理由

1. 宿主团队无法预知未来

VS Code 团队 2015 年发布 1.0 版本时，怎么可能知道 2024 年有人会写一个"用 AI 自动补全代码"的插件？所以他们必须把系统**设计成对未知插件开放**。

2. 插件团队无法修改宿主源码

彩虹猫插件的开发者，不可能去改 VS Code 的源代码把 `new RainbowCatPlugin()` 加进去——他只能按宿主规定的"契约"（接口）来写插件，然后让宿主在运行时发现它。

3. 热插拔需求

你安装一个新插件后，**不想重启 VS Code** 就能用。这就要求宿主能在运行时动态加载 DLL、动态创建实例、动态调用——这正是 23 章讲的全部内容。

真实世界的"宿主 + 加载项"案例

宿主	加载项	接口/契约
VS Code	各种插件	`IPlugin` / `ExtensionContext`
ASP.NET	你写的 Controller、Middleware	`IController`, `IMiddleware`
Unity 游戏引擎	游戏脚本	`MonoBehaviour`
Excel	VBA 宏 / COM 插件	`IDTExtensibility2`
浏览器	扩展程序	各浏览器的 Extension API
游戏 Mod	玩家制作的 Mod	游戏提供的 Mod API

所有这些系统的共同套路都是：

1. **宿主定义一个接口**（契约）
2. **插件实现这个接口**
3. **宿主在运行时扫描、加载、实例化、调用**

回到 23 章：为什么作者要讲这个场景

因为"宿主 + 加载项"是**反射最经典、最有价值的用武之地**。没有反射，这个场景无法实现：

- 没有 `Assembly.LoadFrom` → 没法把未知的 DLL 加载进来
- 没有 `asm.ExportedTypes` → 没法知道 DLL 里有什么类型
- 没有 `Activator.CreateInstance` → 没法动态创建实例
- 没有 `typeof(IPlugin).IsAssignableFrom(type)` → 没法判断类型是否实现了约定接口

四样东西凑齐，才能做到"编译时不知、运行时调用"。

加上 22 章的 AppDomain 后更完整

如果插件有问题（崩溃、内存泄漏、版本冲突），不能影响宿主——所以工业级插件系统会把每个插件加载到**独立的 AppDomain**：

```
宿主 AppDomain（稳定，绝不卸载）
    ↓ 创建
插件 A AppDomain → 加载 PluginA.dll → 实例化 → IPlugin 代理返回给宿主
插件 B AppDomain → 加载 PluginB.dll → 实例化 → IPlugin 代理返回给宿主
```

哪天插件 A 不想要了，`AppDomain.Unload(插件A的域)` 一键清理，干净利落。这就是 22 + 23 章合起来的威力。

一句话总结：宿主就像一个大商场（编译时建好的），加载项就像商户租的店铺（运行时才进驻）。商场建的时候不知道未来会来什么店，但它规定了"所有店铺必须有门、有窗、有招牌"（接口）。商户按规矩装修开业后，顾客（宿主代码）就能通过标准的门走进去消费（调用 `.Activate()`），而不需要关心店里具体卖什么。

这样讲清楚了吗？如果还想看一个**完整可运行的迷你插件系统代码**（包含宿主项目 + 接口项目 + 两个插件项目），我可以给你写一个最小但完整的例子，跑起来你就能亲眼看到"编译时不知、运行时调用"的全过程。

用户:

上面你说的这一部分不太懂，请结合例子解释：

让类型从编译时已知的基类型派生 → 运行时构造实例，引用放到基类变量中，调用基类虚方法

让类型实现编译时已知的接口 → 运行时构造实例，引用放到接口变量中，调用接口方法

元宝:

好，这两个做法是 23.3 节"反射性能优化"的核心。我用同一个例子把两种方式都演示一遍，你对比着看就懂了。

先构造场景

宿主定义了一个"契约"，第三方写了插件。问题是：宿主**编译时不知道插件的具体类名**，怎么高效调用插件的方法？

方式一：基类派生方案

第 **1** 步：宿主定义一个基类（编译时已知）

```
// 宿主程序集里定义
public abstract class PluginBase
{
    public abstract void Activate();

    // 基类可以提供默认实现，派生类可选覆盖
    public virtual void Deactivate()
    {
        Console.WriteLine("默认卸载逻辑");
    }

    public string Name { get; protected set; }
}
```

第 **2** 步：插件继承这个基类

```
// 插件程序集里写（第三方开发者）
public class RainbowCatPlugin : PluginBase
{
    public override void Activate()
    {
        Name = "彩虹猫";
        Console.WriteLine("🌈🐱 激活！");
    }

    // Deactivate 不重写，用基类的默认实现
}
```

第 **3** 步：宿主运行时构造并调用

```
public class PluginHost
{
    public void LoadPlugin(string dllPath)
    {
        Assembly asm = Assembly.LoadFrom(dllPath);

        // 找到继承自 PluginBase 的类型
```

```
            var pluginType = asm.GetTypes()
                .First(t => t.IsSubclassOf(typeof(PluginBase)));

            // 关键：引用放在【基类变量】中
            PluginBase plugin = (PluginBase)Activator.CreateInstance(pluginType);

            // 调用【基类虚方法】—— 这里是编译时已知的调用，很快！
            plugin.Activate();      // CLR 多态派发到 RainbowCatPlugin.Activate
            plugin.Deactivate();    // 调用基类默认实现
        }
    }
```

为什么快？ 因为 `plugin` 变量的类型是 `PluginBase`——这是编译时已知的类型。C# 编译器在编译 `plugin.Activate()` 这行时，就已经知道要调用 `PluginBase.Activate` 这个虚方法，JIT 生成的是标准的虚方法调用指令（callvirt）。**没有运行时字符串查找，没有反射开销。**

方式二：接口实现方案

第 **1** 步：宿主定义一个接口（编译时已知）

```
    // 宿主程序集里定义
    public interface IPlugin
    {
        void Activate();
        void Deactivate();
        string Name { get; }
    }
```

第 **2** 步：插件实现这个接口

```
    // 插件程序集里写（第三方开发者）
    public class RainbowCatPlugin : IPlugin
    {
        public string Name { get; private set; }

        public void Activate()
        {
            Name = "彩虹猫";
            Console.WriteLine("🌈🐱 激活！");
        }

        public void Deactivate()
        {
            Console.WriteLine("彩虹猫卸载");
```

```
        }
    }
```

第 3 步：宿主运行时构造并调用

```
public class PluginHost
{
    public void LoadPlugin(string dllPath)
    {
        Assembly asm = Assembly.LoadFrom(dllPath);

        // 找到实现了 IPlugin 的类型
        var pluginType = asm.GetTypes()
            .First(t => typeof(IPlugin).IsAssignableFrom(t));

        // 关键：引用放在【接口变量】中
        IPlugin plugin = (IPlugin)Activator.CreateInstance(pluginType);

        // 调用【接口方法】—— 同样是编译时已知的调用，很快！
        plugin.Activate();
        plugin.Deactivate();
    }
}
```

为什么快？ 同理，`plugin` 变量的类型是 `IPlugin`——编译时已知。调用 `plugin.Activate()` 时，JIT 生成的是标准的接口方法调度指令，**没有反射开销**。

对比：如果不这样做，用纯反射调用有多慢

```
// ❌ 慢速方案：纯反射调用
public void LoadPluginSlow(string dllPath)
{
    Assembly asm = Assembly.LoadFrom(dllPath);
    var pluginType = asm.GetTypes()[0];

    object instance = Activator.CreateInstance(pluginType);  // 返回 object

    // 用反射找方法 —— 运行时字符串查找，慢！
    MethodInfo activateMethod = pluginType.GetMethod("Activate");

    // 用反射调用 —— 实参要打包成数组，内部再解包，还要检查权限，慢！
    activateMethod.Invoke(instance, null);
}
```

慢在三处：

1. `GetMethod("Activate")`：要在元数据表里做字符串匹配查找
2. `Invoke(instance, null)`：实参要打包成 `object[]` 数组 → CLR 内部解包到线程栈 → 检查实参类型 → 检查调用者权限
3. **每次调用都要重复以上过程**

而用基类/接口方案：

```
IPlugin plugin = (IPlugin)Activator.CreateInstance(pluginType);
plugin.Activate();  // 直接 callvirt 指令，一步到位
plugin.Activate();  // 第二次调用同样快，零反射开销
plugin.Activate();  // 第 N 次还是快
```

`Activator.CreateInstance` 本身也有一点开销，但**只发生一次**。之后的 `plugin.Activate()` 调用全是标准的虚方法/接口方法调度，和直接写 `new RainbowCatPlugin().Activate()` 几乎一样快。

基类方案 vs 接口方案：怎么选？

维度	基类方案	接口方案
插件能继承其他类吗?	❌ 不能（C# 单继承）	✅ 能（可以同时实现多个接口）
能提供默认实现吗?	✅ 能（虚方法）	❌ 不能（C# 8.0 前；C# 8.0+ 有默认接口方法但有限制）
版本控制	基类加成员，派生类自动继承，**兼容性好**	接口加成员，所有实现者必须改代码，**破坏性变更**
灵活性	限制插件必须从 PluginBase 派生	插件类可以自由设计，只要实现 IPlugin 即可
作者偏好	-	✅ 本书作者更偏好接口方案

具体选择建议：

用基类方案：

- 宿主需要提供大量默认行为，插件只需要覆盖一小部分
- 预计将来会在契约里加新成员，希望老插件不用改代码就能兼容
- 例如：游戏引擎的 `MonoBehaviour`、ASP.NET 的 `ControllerBase`

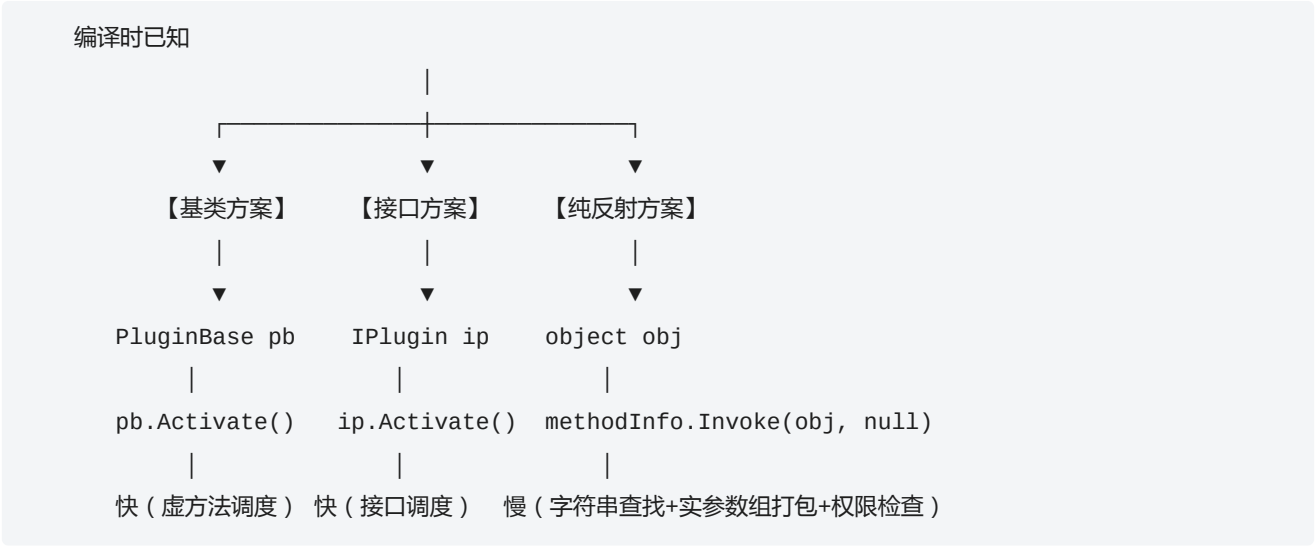
用接口方案：

- 契约简单，只是一组方法签名
- 插件开发者希望自己的类能自由继承其他类
- 契约稳定，不会频繁加新成员
- 例如：ASP.NET 的 `IMiddleware`、日志库的 `ILogger`

> 💡 本书作者的实战经验：接口方案更灵活，因为基类方案会"污染"插件的继承体系——插件开发者可能本来就想让自己的插件继承自别的类。但涉及版本控制时，基类方案更稳，因为往基类加个虚方法，所有派生类自动继承，而往接口加个方法，所有实现者都得改代码重编译。

> **最佳实践**：把接口/基类放到**独立的程序集**中（比如 `MyApp.PluginContracts.dll`），宿主和插件都引用它。这样契约的版本演进不会强制要求宿主和插件同时重新编译。

一图总结



两种方式都把"运行时才知道的具体类型"向上转型为"编译时已知的基类/接口"，从而把**昂贵的反射调用**转化为**廉价的虚方法/接口方法调度**。这就是 23.3 节性能优化的精髓。

要不要我把这两种方案写成一个**完整可运行的 mini 插件系统**（宿主项目 + 契约程序集 + 两个插件项目），你 clone 下来跑一下，亲眼看看 `Activator.CreateInstance` + 接口转型后的调用有多流畅？这样能把 22、23 章的知识串成一个能跑的东西。

用户:

上面你提到的这两个不太懂，请结合例子解释：

23.3.2 类型对象的准确含义

程序集中每个类型只有一个 Type 对象，所以可以用 == 判断两个 Type 引用是否指向同一类型 。

23.3.3 构建 Exception 派生类型的一个层次结构

通过一个具体例子演示如何用反射发现类型并构建异常类型的层次结构（用于动态展示/处理）。

元宝:

这两个概念一个是**底层机制**（类型对象在内存里长什么样），一个是**实际应用**（用反射去"摸底"所有异常类型）。我分别用大白话和代码例子给你讲透。

23.3.2 类型对象的准确含义

先搞懂：什么是"类型对象"？

你写的每一个类（比如 `Person`），在程序运行起来之后，CLR 都会在内存里为它创建**一个专门的数据结构**，用来存储这个类的所有信息：它叫什么名字、有什么字段、有什么方法、父类是谁、实现了哪些接口……

这个数据结构，就是 **类型对象（Type Object）**。我们在 C# 代码里通过 `typeof(Person)` 拿到的那个 `System.Type` 对象，就是对这个内部数据结构的"托管包装"。

核心规则：每种类型只有一个 Type 对象

不管你在程序的哪个角落、用哪种方式去获取同一个类型的 `Type` 对象，**拿到的永远是同一个引用。**

用一个生活例子比喻：

> 户籍系统里，张三这个人只有一条档案记录。你去派出所查（用身份证号查）、去居委会查（用户口本查）、网上查（用姓名+身份证号查），查出来的都是**同一条档案**，而不是复印了三份。

代码验证

```
using System;

class Person { }

class Program
{
    static void Main()
    {
        Person p1 = new Person();
        Person p2 = new Person();

        // 方式1：用 typeof 运算符
        Type t1 = typeof(Person);

        // 方式2：用实例的 GetType() 方法
        Type t2 = p1.GetType();
        Type t3 = p2.GetType();

        // 方式3：用字符串全名（包含命名空间）
        Type t4 = Type.GetType("Person");

        Console.WriteLine($"t1 == t2 : {t1 == t2}");   // True
        Console.WriteLine($"t2 == t3 : {t2 == t3}");   // True
        Console.WriteLine($"t1 == t4 : {t1 == t4}");   // True（如果命名空间匹配）

        // 最严格的引用比较
        Console.WriteLine($"引用完全相同: {object.ReferenceEquals(t1, t2)}"); // True
        Console.WriteLine($"t1 的 HashCode: {t1.GetHashCode()}");
        Console.WriteLine($"t2 的 HashCode: {t2.GetHashCode()}"); // 一模一样
    }
}
```

这个特性有什么用？

正因为 Type 对象是唯一的，所以你可以放心地用它做这些事：

用途	说明
`==` **判断类型**	`if (obj.GetType() == typeof(Person))` —— 比 `is` 运算符更严格（不考虑继承），而且因为是引用比较，速度极快
作为字典的 Key	`Dictionary<Type, object> cache` —— 依赖注入容器、序列化器经常这么干，因为同一个类型的 Type 对象一定相等，HashCode 也稳定
作为 lock 对象	`lock(typeof(Person))` —— 虽然不推荐（可能被人从外部 lock 导致死锁），但技术上可行，因为全局唯一

一个重要的延伸：泛型类型

"每种类型只有一个"也适用于泛型。但注意：**泛型参数不同，就是不同的类型**：

```
Type listInt = typeof(List<int>);
Type listString = typeof(List<string>);

Console.WriteLine(listInt == listString);  // False！这是两个不同的 Type 对象
Console.WriteLine(listInt.FullName);       // System.Collections.Generic.List`1[[Syste
m.Int32]]
Console.WriteLine(listString.FullName);    // System.Collections.Generic.List`1[[Syste
m.String]]
```

CLR 把 `List<int>` 和 `List<string>` 视为完全不同的两种类型，各自有独立的 Type 对象、独立的 JIT 编译代码、独立的内存布局。

23.3.3 构建 Exception 派生类型的一个层次结构

这节到底在干什么？

.NET 基础类库（mscorlib / System.Private.CoreLib）里定义了**上百种**异常类型，它们之间有着复杂的继承关系：

```
Exception
 ├── SystemException
 │    ├── ArgumentException
 │    │    ├── ArgumentNullException
 │    │    └── ArgumentOutOfRangeException
 │    ├── InvalidOperationException
 │    └── NullReferenceException
 └── ApplicationException
      └── YourCustomException
```

23.3.3 节的目的就是：**写一段代码，自动扫描程序集，把这棵"异常家族树"挖出来，构建成结构化的数据。**这在写日志框架、IDE 工具、或者动态异常处理中间件时非常有用。

核心思路

1. 拿到包含 `Exception` 类的程序集（ `typeof(Exception).Assembly` ）
2. 遍历里面所有的类型
3. 筛选出所有继承自 `Exception` 的类型
4. 利用 `Type.BaseType` 属性理清父子关系，构建成一棵树

完整示例代码

```
using System;
using System.Collections.Generic;
using System.Linq;

class Program
{
    static void Main()
    {
        // 第1步：拿到 Exception 所在的程序集
        Assembly coreAssembly = typeof(Exception).Assembly;

        // 第2步：找出所有具体的（非抽象）Exception 派生类
        var allExceptions = coreAssembly.GetTypes()
            .Where(t => t.IsClass
                     && !t.IsAbstract
                     && typeof(Exception).IsAssignableFrom(t))
            .ToList();

        Console.WriteLine($"共找到 {allExceptions.Count} 种异常类型\n");

        // 第3步：构建父子关系字典（层次结构）
        // Key = 父类型, Value = 直接子类列表
        Dictionary<Type, List<Type>> hierarchy = new Dictionary<Type, List<Type>>();

        foreach (var exType in allExceptions)
        {
            Type parent = exType.BaseType;
            if (parent == null) continue;

            if (!hierarchy.ContainsKey(parent))
                hierarchy[parent] = new List<Type>();

            hierarchy[parent].Add(exType);
```

```
            }

            // 第4步：从 Exception 开始，递归打印整棵树
            PrintTree(typeof(Exception), hierarchy, 0);

            // 额外演示：打印某个具体类型的完整继承链
            Console.WriteLine("\n===== 单个类型的继承链 =====");
            PrintInheritanceChain(typeof(ArgumentNullException));
        }

        /// <summary>
        /// 递归打印异常层次树
        /// </summary>
        static void PrintTree(Type parent, Dictionary<Type, List<Type>> tree, int depth)
        {
            Console.WriteLine($"{new string(' ', depth * 2)}{parent.Name}");

            if (tree.ContainsKey(parent))
            {
                foreach (var child in tree[parent].OrderBy(t => t.Name))
                {
                    PrintTree(child, tree, depth + 1);
                }
            }
        }

        /// <summary>
        /// 打印某个类型的完整继承链（从自身一直到 object）
        /// </summary>
        static void PrintInheritanceChain(Type type)
        {
            Type current = type;
            while (current != null)
            {
                Console.WriteLine(current.FullName);
                Console.WriteLine("  ↑ 继承自");
                current = current.BaseType;
            }
            Console.WriteLine("System.Object");
        }
    }
```

运行效果（节选）

```
Exception
  SystemException
    ArgumentException
      ArgumentNullException
      ArgumentOutOfRangeException
      DuplicateWaitObjectException
    InvalidOperationException
      ObjectDisposedException
    NullReferenceException
  ApplicationException
    TargetInvocationException
```

单个继承链的输出：

```
System.ArgumentNullException
  ↑ 继承自
System.ArgumentException
  ↑ 继承自
System.SystemException
  ↑ 继承自
System.Exception
  ↑ 继承自
System.Object
```

这段代码用到了哪些 **23** 章的知识？

代码中的操作	对应的 23 章知识点
`typeof(Exception).Assembly`	23.1 程序集加载（获取已加载的程序集）
`coreAssembly.GetTypes()`	23.3.1 发现程序集中定义的类型
`type.IsClass`，`!type.IsAbstract`	23.5.1/23.5.2 用 BindingFlags 和 Type 属性筛选成员
`typeof(Exception).IsAssignableFrom(t)`	23.3.2 类型对象的比较与判定（判断继承关系）
`type.BaseType`	23.3.3 构建层次结构的核心（获取直接基类）

为什么要"构建"而不是直接写死？

因为 .NET 版本在不断演进，每个新版本都可能增加新的异常类型（比如 .NET 6 加了 `System.Diagnostics.UnreachableException`）。如果你手写一棵固定的树，新异常就漏掉了。**用反射动态构建，无论 .NET 怎么升级，你的代码都能自动感知到所有异常类型。**这就是反射的强大之处——它让你的程序具备了"自我探索"的能力。

两节之间的联系

23.3.2 和 23.3.3 其实是**因果关系**：

- **23.3.2（因）**：因为每种类型只有一个 Type 对象，所以 `Type` 可以安全地作为字典的 Key、可以可靠地用 `==` 比较。
- **23.3.3（果）**：正因为有这种可靠性，我们才能放心地用 `Dictionary<Type, List<Type>>` 把异常类型的父子关系存起来，不用担心 Key 冲突或比较出错。

如果你愿意，下一步我可以帮你把 23.3.4（构造类型的实例）和 23.5（发现成员）也用这种"大白话+可运行代码"的方式过一遍，这样整章的反射知识就串成一条线了。

用户:

你上面解释的这部分不太懂，请结合例子解释：

23.5.4 调用类型的成员

23.5.6 使用绑定句柄来减少进程的内存耗用

元宝:

这两节其实都在讲**反射的性能优化**，只不过角度不同：23.5.4 优化的是"调用速度"，23.5.6 优化的是"内存占用"。我用代码例子把每一步都展开讲。

23.5.4 调用类型的成员

问题背景

假设你有一个类型，需要通过反射来调用它的方法。有几种做法，性能差别巨大。

先准备一个测试用的类：

```
public class Calculator
{
    public int Add(int a, int b) => a + b;

    public string Name { get; set; } = "MyCalculator";

    public void PrintSum(int x, int y)
    {
        Console.WriteLine($"{x} + {y} = {x + y}");
    }
}
```

方式一：`Type.InvokeMember`（最慢）

```
Type type = typeof(Calculator);
object instance = Activator.CreateInstance(type);

// 用 InvokeMember 调用 Add 方法
object result = type.InvokeMember(
    "Add",                                                  // 方法名（字符串）
    BindingFlags.InvokeMethod | BindingFlags.Public | BindingFlags.Instance,
    null,                                                   // Binder（用默认）
    instance,                                               // 目标对象
    new object[] { 1, 2 }                                   // 实参数组
);

Console.WriteLine(result); // 3
```

为什么最慢？

`InvokeMember` 相当于"一站式服务"——它内部要依次做这些事：

1. 根据方法名 `"Add"` 在元数据表里**搜索**匹配的方法（字符串比对）
2. 根据 `BindingFlags` **筛选**出正确的方法
3. 如果找到了多个重载（比如 `Add(int, int)` 和 `Add(double, double)`），还要做**重载决议**（选最匹配的）
4. 把实参数组里的元素**逐个类型检查**
5. 最后才调用

而且**每次调用都要重复以上全部步骤**。如果你在一个循环里调用 10000 次，搜索就做了 10000 次。

方式二：`MethodInfo.Invoke`（中等）

```
Type type = typeof(Calculator);
object instance = Activator.CreateInstance(type);

// 先"绑定"一次：找到 MethodInfo
MethodInfo addMethod = type.GetMethod("Add");

// 然后多次调用
for (int i = 0; i < 10000; i++)
{
    object result = addMethod.Invoke(instance, new object[] { i, i + 1 });
}
```

为什么比方式一快？

关键在于 **"一次绑定，多次调用"**：

- `type.GetMethod("Add")` —— 这一步做了搜索、筛选、重载决议，拿到了一个 `MethodInfo` 对象
- 之后每次 `addMethod.Invoke(...)` —— **不需要再搜索了**，直接调用

搜索只做一次，所以比 `InvokeMember` 快。但 `Invoke` 本身仍然有开销：实参要打包成数组、CLR 要检查类型安全、要检查调用权限。

方式三：创建委托（更快）

```
Type type = typeof(Calculator);
object instance = Activator.CreateInstance(type);

// 先获取 MethodInfo（绑定一次）
MethodInfo addMethod = type.GetMethod("Add");

// 把 MethodInfo 包装成强类型委托
var addDelegate = (Func<int, int, int>)Delegate.CreateDelegate(
    typeof(Func<int, int, int>),  // 委托类型
    instance,                     // 目标对象
    addMethod                     // 方法信息
);

// 调用委托 —— 几乎没有反射开销
for (int i = 0; i < 10000; i++)
{
    int result = addDelegate(i, i + 1);  // 直接调用，像普通方法一样快
}
```

为什么更快？

委托是 CLR 的一等公民。创建委托后，CLR 内部生成一个"跳板"（thunk），直接把调用转发到目标方法。调用委托时：

- 不需要再查元数据表
- 不需要打包实参数组
- 不需要做类型安全检查（编译时已经检查过了）
- JIT 甚至可以内联这个小调用

性能接近直接写 `calculator.Add(1, 2)`。

方式四：`dynamic`（C# 4.0+，语法最简）

```
using System.Dynamic;

Type type = typeof(Calculator);
object instance = Activator.CreateInstance(type);

// 转为 dynamic
dynamic calc = instance;
```

```
// 像普通对象一样调用
calc.PrintSum(1, 2);  // DLR 在运行时解析成员
```

原理：`dynamic` 背后是 DLR（Dynamic Language Runtime）。第一次调用 `calc.PrintSum` 时，DLR 会：

1. 缓存调用站点（Call Site）
2. 做一次成员查找和方法绑定
3. 后续调用直接从缓存中取，**不再重复绑定**

所以性能和 `MethodInfo.Invoke`（方式二）接近，但代码写起来最简洁。

四种方式性能对比总结

方式	相对速度	绑定次数	适用场景
`Type.InvokeMember`	🐢 最慢	每次调用都重新绑定	偶尔调用一次，代码量少
`MethodInfo.Invoke`	🚶 中等	绑定一次	需要多次调用同一成员
创建委托	🏃 很快	绑定一次	性能敏感的热路径
`dynamic`	🚶 中等偏快	DLR 缓存	代码简洁优先，性能次之

23.5.6 使用绑定句柄来减少进程的内存耗用

问题背景

假设你正在写一个工具，需要扫描程序集里**所有类型、所有方法**，并缓存它们的信息以便后续使用。

直觉写法：

```
// 缓存所有方法的 MethodInfo 对象
Dictionary<string, MethodInfo> methodCache = new Dictionary<string, MethodInfo>();

foreach (Type type in Assembly.Load("MyLib").GetTypes())
{
    foreach (MethodInfo method in type.GetMethods())
    {
        methodCache[$"{type.Name}.{method.Name}"] = method;
    }
}
```

问题：`MethodInfo` 是个"胖对象"。

为什么 `MethodInfo` 占用内存大？

一个 `MethodInfo` 对象内部引用了一整条对象链：

```
MethodInfo
 ├── Name (string)           ← 字符串对象
 ├── DeclaringType (Type)    ← 指向类型对象
 │    ├── FullName (string)
 │    ├── Assembly (Assembly) ← 指向程序集对象
 │    ├── BaseType (Type)
 │    └── ...
 ├── ReturnType (Type)       ← 又是一个 Type 对象
 ├── Parameters (ParameterInfo[]) ← 参数数组
 │    ├── ParameterInfo[0]
 │    │    ├── Name (string)
 │    │    ├── ParameterType (Type)
 │    │    └── ...
 │    └── ParameterInfo[1]
 │         └── ...
 ├── CustomAttributes (object[]) ← 自定义特性
 └── ...
```

扫描一个大型程序集（比如 `System.Private.CoreLib`），可能有上万种类型、几十万个方法。如果每个 `MethodInfo` 连带它引用的整个对象图都常驻内存，内存占用会非常可观。

解决方案：用"绑定句柄"代替"MemberInfo 对象"

CLR 提供了三种轻量级句柄类型：

句柄类型	对应信息	获取方式
`RuntimeTypeHandle`	类型信息	`typeof(T).TypeHandle` 或 `type.TypeHandle`
`RuntimeMethodHandle`	方法信息	`method.MethodHandle`
`RuntimeFieldHandle`	字段信息	`field.FieldHandle`

句柄本质上是一个**轻量级的值类型**（内部就是一个指针/令牌），占用内存极小（通常 4~8 字节），而且**不持有对 Type、Assembly 等大对象的引用。**

代码示例

```
using System;
using System.Reflection;

public class Person
{
    public string Name { get; set; }
    public int Age { get; set; }

    public void SayHello() => Console.WriteLine($"Hello, I'm {Name}");
```

```
    public int GetBirthYear(int currentYear) => currentYear - Age;
}

class Program
{
    static void Main()
    {
        Type type = typeof(Person);

        // ===== 获取句柄 =====

        // 类型句柄
        RuntimeTypeHandle typeHandle = type.TypeHandle;
        Console.WriteLine($"TypeHandle: {typeHandle}");

        // 方法句柄
        MethodInfo sayHelloMethod = type.GetMethod("SayHello");
        RuntimeMethodHandle methodHandle = sayHelloMethod.MethodHandle;

        // 属性句柄（通过属性的 getter/setter 方法获取）
        PropertyInfo nameProperty = type.GetProperty("Name");
        RuntimeMethodHandle getterHandle = nameProperty.GetGetMethod().MethodHandle;

        // ===== 从句柄还原 =====

        // 从句柄还原回 Type 对象
        Type restoredType = Type.GetTypeFromHandle(typeHandle);
        Console.WriteLine($"还原的类型名: {restoredType.Name}"); // Person
        Console.WriteLine($"类型相同: {restoredType == type}");   // True

        // 从句柄还原回 MethodInfo 对象
        MethodInfo restoredMethod = (MethodInfo)MethodInfo.GetMethodFromHandle(methodHa
ndle);
        Console.WriteLine($"还原的方法名: {restoredMethod.Name}"); // SayHello

        // ===== 实际使用：用句柄做缓存 =====

        // 用句柄做 Dictionary 的 Key（比用 Type/MethodInfo 更省内存）
        Dictionary<RuntimeMethodHandle, string> methodCache =
            new Dictionary<RuntimeMethodHandle, string>();

        foreach (MethodInfo method in type.GetMethods(BindingFlags.Public | BindingFlag
s.Instance | BindingFlags.DeclaredOnly))
        {
```

```
            methodCache[method.MethodHandle] = method.Name;
        }

        Console.WriteLine($"\n缓存了 {methodCache.Count} 个方法句柄");

        // 需要时再还原
        foreach (var kvp in methodCache)
        {
            MethodInfo m = (MethodInfo)MethodInfo.GetMethodFromHandle(kvp.Key);
            Console.WriteLine($"  {m.Name}");
        }
    }
}
```

什么时候该用句柄？

场景	用句柄	用 Type/MemberInfo
临时反射操作（用完即弃）	❌	✅ 方便
长期缓存大量类型/成员信息	✅ 省内存	❌ 内存压力大
需要频繁访问成员元数据	❌ 每次还原有开销	✅ 直接访问
只做类型比较/查找	✅ 轻量	❌ 杀鸡用牛刀

一个更实际的例子：**ORM** 框架的元数据缓存

假设你写一个微型 ORM，需要缓存每个实体类的属性映射信息：

```
// ❌ 不好的做法：直接缓存 PropertyInfo
class BadEntityMapper
{
    // PropertyInfo 是大对象，1000 个实体类 × 10 个属性 = 10000 个大对象常驻内存
    Dictionary<Type, PropertyInfo[]> _cache = new Dictionary<Type, PropertyInfo[]>();
}

// ✅ 好的做法：缓存句柄
class GoodEntityMapper
{
    // 句柄很小，只存值类型
    Dictionary<RuntimeTypeHandle, RuntimePropertyHandle[]> _cache =
        new Dictionary<RuntimeTypeHandle, RuntimePropertyHandle[]>();

    public void CacheProperties<T>()
    {
        Type type = typeof(T);
```

```
            var handles = type.GetProperties()
                .Select(p => p.PropertyHandle)
                .ToArray();

            _cache[type.TypeHandle] = handles;
        }

        public void MapToDb<T>(T entity)
        {
            var handles = _cache[typeof(T).TypeHandle];

            foreach (var handle in handles)
            {
                // 需要时再还原
                PropertyInfo prop = PropertyInfo.GetPropertyFromHandle(handle);
                var value = prop.GetValue(entity);
                // 写入数据库...
            }
        }
    }
```

两节的联系

23.5.4 和 23.5.6 其实是在解决反射的两个不同维度的性能问题：

维度	23.5.4 解决什么	23.5.6 解决什么
时间（CPU）	调用成员的速度	不涉及（还原句柄反而多一步）
空间（内存）	不涉及	缓存大量元数据时的内存占用

最佳实践是组合使用：

1. 用 **句柄** 做长期缓存（省内存）
2. 需要频繁调用时，从句柄还原出 `MethodInfo`，再创建**委托**（省 CPU）
3. 偶尔调用时，直接从句柄还原出 `MethodInfo` 用 `Invoke`（平衡内存和速度）

```
    // 终极优化方案示意
    RuntimeMethodHandle methodHandle = ...;  // 从缓存取句柄

    // 第一次：从句柄还原 + 创建委托
    MethodInfo method = MethodInfo.GetMethodFromHandle(methodHandle) as MethodInfo;
    var delegate = Delegate.CreateDelegate(typeof(Action), target, method);
```

```
    // 后续：直接调用委托（最快）
    delegate.DynamicInvoke();  // 或用强类型委托
```

一句话总结：

- **23.5.4**：反射调用成员时，别用 `InvokeMember`（每次都重新搜索），用 `MethodInfo.Invoke`（一次绑定多次调用），最好创建委托（接近原生速度）
- **23.5.6**：如果要长期缓存大量类型/成员信息，别直接存 `Type` / `MethodInfo`（太胖），存它们的**句柄**（轻量值类型），需要时再还原

这样讲清楚了吗？如果还想看某个具体场景的完整代码（比如结合 22 章 AppDomain 的跨域反射调用），告诉我，我继续展开。

用户:

“元数据”是跟随程序集的，所以某一个类型的反射使用底层都是使用该类型所属的程序集的元数据，即首先得加载整个程序集；这样理解对吗？

元宝:

你的理解**大方向完全正确**，而且抓住了反射最核心的本质。我帮你把这个认知再精确化一层，顺便澄清一个常见的微小误区。

你的理解哪里是对的

> **"元数据跟随程序集，反射底层就是读元数据，所以必须先加载整个程序集。"**

这三点完全正确，而且是 23 章的基石：

1. **元数据是程序集的一部分** —— 不能脱离程序集单独存在
2. **反射的本质 = 读取元数据表** —— `Type.GetMethods()` 就是扫 `MethodDef` 表
3. **程序集是加载的最小单元** —— CLR 没有提供"只加载某一个类型"的 API，你必须把整个程序集加载进来

需要精确化的一点："加载整个程序集"到底加载了什么？

你说"首先得加载整个程序集"，这里的"加载"需要区分两层含义，不然容易误以为"加载程序集 = 把所有代码都执行了一遍"：

第一层：元数据 + IL 被映射到内存（确实加载了"整个"）

当你调用 `Assembly.Load("MyLib")` 时，CLR 会：

- 找到 `MyLib.dll` 文件
- 把 PE 文件头、清单、所有元数据表、**所有 IL 代码** 都映射到进程内存中

- 如果有依赖项（比如 `MyLib` 引用了 `Newtonsoft.Json`），CLR 会**递归加载**依赖项的元数据（这也是为什么有时候你只反射了一个类型，却看到一堆相关程序集被加载进来）

到这一步为止，程序集里的 **代码一行都没执行**，只是"说明书"被翻开了。

第二层：类型和方法不一定被"初始化"

- **类型对象（Type Object）**：通常在第一次用到该类型时才创建（比如第一次 `typeof(T)` 或 `new T()`），不是 `Assembly.Load` 瞬间就创建好所有类型的 Type 对象
- **静态构造函数**：在类型第一次被访问时触发，不是程序集加载时
- **JIT 编译**：方法的 IL 代码在**第一次被调用时**才被 JIT 编译成本地机器码，不是加载时就编译

所以更准确的说法是：

> **"反射一个类型，必须先让包含它的程序集的元数据进入内存。但这只是把'说明书'翻开，不代表把书里所有的'动作'都做了一遍。"**

用一个完整例子验证你的理解

```
// MyLib.dll 中的代码
public class UserService
{
    static UserService()
    {
        Console.WriteLine("UserService 静态构造函数被调用了！");
    }

    public void PrintUser() => Console.WriteLine("用户数据");
}

public class OrderService
{
    static OrderService()
    {
        Console.WriteLine("OrderService 静态构造函数被调用了！");
    }

    public void PrintOrder() => Console.WriteLine("订单数据");
}
```

```
// 宿主程序
class Program
{
    static void Main()
    {
```

```
            Console.WriteLine("1. 开始加载程序集");
            Assembly asm = Assembly.Load("MyLib");
            Console.WriteLine("2. 程序集加载完毕");

            // 此时 MyLib 已经"整个"加载进来了
            // 但两个静态构造函数都还没触发

            Console.WriteLine("3. 开始反射查找类型");
            Type userServiceType = asm.GetType("UserService");
            Console.WriteLine($"4. 找到了类型: {userServiceType.Name}");
            // 到这里，OrderService 的静态构造函数也没触发

            Console.WriteLine("5. 创建 UserService 实例");
            object instance = Activator.CreateInstance(userServiceType);
            // 此时控制台才会输出："UserService 静态构造函数被调用了！"
            // OrderService 的静态构造函数仍然没触发
        }
    }
```

运行输出：

```
1. 开始加载程序集
2. 程序集加载完毕
3. 开始反射查找类型
4. 找到了类型: UserService
5. 创建 UserService 实例
UserService 静态构造函数被调用了！
```

这个例子验证了：

- ✅ 程序集确实被整体加载了（元数据全在内存里）
- ✅ 但只有你真正用到某个类型时，它才被"激活"
- ✅ 没用到的类型（`OrderService`）即使在同一程序集中，也安静地躺在内存里不动

再补充一个你可能会想到的延伸问题

"如果我只需要一个类型，能不能只加载这一个类型，不加载整个程序集？"

答案是不能。 原因是元数据表之间有交叉引用：

```
TypeDef 表（类型定义）
  → 第 5 行记录：UserService 类
      → 父类列：指向 TypeRef 表的第 3 行（object）
      → 方法列：指向 MethodDef 表的第 12~15 行
```

```
TypeRef 表（类型引用）
  → 第 3 行：System.Object
      → 指向 AssemblyRef 表的第 1 行（mscorlib）
```

你要看 `UserService` 的信息，CLR 必须能顺着这些表链一路追下去——追到父类、追到方法签名里的参数类型、追到引用了哪些其他程序集。所以**必须加载整个程序集的元数据**，没办法只读其中一小块。

一句话总结你的理解

> **"程序集是元数据的载体，也是 CLR 加载的最小单元。反射一个类型的前提是它所属的程序集已被加载，也就是它的元数据已经在内存中了。但加载 ≠ 执行，只是让 CLR 能'看见'类型的描述信息。"**

你的理解已经非常到位了。这其实就是 23.1 节 `Assembly.Load` 和 23.2 节反射之间那条隐形的连线——**没有程序集加载，就没有元数据可读，也就没有反射可言。**

如果想继续往下挖，下一个值得思考的问题是："既然加载了整个程序集，那能不能只卸载其中一个类型？"——这个问题会自然地把你引回 22 章的 AppDomain 知识 😉

用户:

对于串联22章和23章的知识方面有点疑惑，我们正常运行一个exe是加载clr=》创建appdomain=》如何如何加载程序集，这个加载程序集需要我们显式地写代码实现吗？

元宝:

直接回答：正常情况下你**不需要**显式写代码

双击一个 `.exe` 运行，整个过程完全是**自动**的，你一行加载代码都不用写。CLR 的加载、AppDomain 的创建、程序集的加载，全由系统自动完成。

自动加载的完整流程

当你双击 `MyApp.exe` 时，Windows 做了这些事：

```
1. Windows 创建进程
2. 检查 PE 头，发现是 .NET 程序
3. 加载 mscoree.dll（CLR 的引导 DLL）
4. mscoree 启动 CLR（相当于 22.1 节的"CLR 寄宿"）
5. CLR 创建默认 AppDomain（相当于 22.2 节）
6. CLR 加载 MyApp.exe 这个程序集本身（相当于 23.1 节）
7. 检查 MyApp.exe 的 AssemblyRef 元数据表
8. 递归加载所有被引用的程序集
9. 执行 Main() 方法
```

这一切都不需要你写任何代码——CLR 的启动代码（也叫"垫片/shim"）帮你全做了。

用代码验证"自动加载"

```
// Program.cs —— 你平时写的代码
class Program
{
    static void Main()
    {
        // 这一行代码运行时，以下事情早已经自动发生完了：
        // ✅ CLR 已启动
        // ✅ 默认 AppDomain 已创建
        // ✅ MyApp.exe 程序集已加载
        // ✅ 所有被引用的程序集（比如 System.Runtime）已加载

        Console.WriteLine(AppDomain.CurrentDomain.FriendlyName);
        // 输出: "MyApp.exe" —— 这就是默认 AppDomain

        // 看看当前 AppDomain 里已经加载了哪些程序集
        foreach (Assembly asm in AppDomain.CurrentDomain.GetAssemblies())
        {
            Console.WriteLine(asm.GetName().Name);
        }
        // 你会看到：MyApp, System.Runtime, System.Console,
        //          System.Private.CoreLib ... 等等
        // 这些都是自动加载的，你没写一行 Assembly.Load
    }
}
```

那 23 章讲的 `Assembly.Load` 是干嘛用的？

这是关键问题。`Assembly.Load` **不是用来加载"正常引用的程序集"的**，那种情况 CLR 自动就做了。

`Assembly.Load` 是为以下**特殊场景**准备的：

场景 1：插件系统（最常见的用途）

```
// 宿主程序编译时，根本不知道插件 DLL 的存在
// 所以没法通过"添加引用"让 CLR 自动加载
// 必须在运行时动态加载

string pluginPath = Path.Combine(AppDomain.CurrentDomain.BaseDirectory, "plugins", "MyP
lugin.dll");
Assembly pluginAsm = Assembly.LoadFrom(pluginPath);  // ← 这里必须显式调用
```

场景 2：延迟加载 / 按需加载

```
// 某些程序集很大，启动时加载会拖慢速度
// 所以只在用户真正用到时才加载

public void OnUserClickExportButton()
{
    // 用户点击"导出"按钮时才加载 Excel 库
    Assembly excelAsm = Assembly.Load("Office.Excel.Interop");
    // 然后反射创建相关类型...
}
```

场景 **3**：按条件加载不同实现

```
// 根据配置决定加载哪个数据库驱动
Assembly dbAsm;
if (config.DatabaseType == "MySQL")
    dbAsm = Assembly.Load("MySql.Data");
else if (config.DatabaseType == "PostgreSQL")
    dbAsm = Assembly.Load("Npgsql");
```

场景 **4**：嵌入式资源程序集

```
// DLL 被嵌进 exe 里，运行时从内存流加载
byte[] dllBytes = GetEmbeddedResource("MyLib.dll");
Assembly asm = Assembly.Load(dllBytes);  // ← 显式调用
```

把 **22** 章和 **23** 章串起来看

```
┌──────────────────────────────────────────────────────┐
│  双击 MyApp.exe                                      │
│                                                      │
│  ── 自动阶段（Windows + CLR 启动代码完成）──          │
│  ① 加载 CLR（寄宿）                    [22.1]        │
│  ② 创建默认 AppDomain                  [22.2]        │
│  ③ 加载 MyApp.exe 程序集               [23.1]        │
│  ④ 递归加载所有静态引用的程序集        [23.1]        │
│  ⑤ 执行 Main()                                       │
│                                                      │
│  ── 手动阶段（你的代码里写）──                       │
│  ⑥ 如果需要插件/动态加载：                           │
│     Assembly.LoadFrom("plugin.dll")    [23.1]        │
│     → 程序集被加载到当前 AppDomain                   │
│  ⑦ 反射发现类型                        [23.2~23.5]   │
│  ⑧ 动态创建实例并调用                  [23.3.4]      │
```

```
│                                                          │
│  ── 高级玩法（结合 22 章）──                               │
│  ⑨ 创建独立 AppDomain 隔离插件                  [22.2]    │
│     AppDomain pluginDomain = AppDomain.CreateDomain();   │
│  ⑩ 在 pluginDomain 中加载插件程序集            [23.1]    │
│  ⑪ 不用了就卸载整个 AppDomain                  [22.3]    │
│     AppDomain.Unload(pluginDomain);                      │
└──────────────────────────────────────────────────────────┘
```

一个关键认知

> **CLR 的自动加载解决的是"已知依赖"的问题，23 章的 `Assembly.Load` 解决的是"未知/动态"的问题。**

- **已知依赖**：你在 Visual Studio 里"添加引用" → 编译进 AssemblyRef 表 → CLR 启动时自动加载
- **未知/动态**：插件、延迟加载、条件加载 → 必须显式调用 `Assembly.Load`

这就是为什么普通的 WinForms/WPF/Console 程序里你从来没写过 `Assembly.Load`——因为所有依赖都是编译时已知的，CLR 自动搞定。只有当你要做"插件系统"、"热更新"、"动态扩展"这类高级架构时，才需要显式调用 23 章的那些 API。

一句话总结

> **正常运行 exe 时，CLR 启动、AppDomain 创建、程序集加载——全自动，你啥都不用写。23 章的 `Assembly.Load` 等方法，是给"需要在运行时动态加载未知程序集"的场景准备的武器。普通开发用不到，但做插件架构、可扩展系统设计时必须掌握。**

这也是为什么 23 章开头就说"反射的典型场景是宿主 + 加载项"——因为只有在这种场景下，你才需要手动干预程序集的加载过程。日常开发中 CLR 的自动加载机制已经够用了。

（注：文档部分内容可能由 AI 生成）